Earendil: Um Sistema Baseado em *Machine Learning* para Detecção de Falhas em Pavimentos Urbanos por Meio de Drones

Fabrício José[1], **Marco Abbud**[1]

[1]Centro Federal de Educação Tecnológica Celso Suckow da Fonseca (CEFET/RJ)
Rio de Janeiro, RJ, Brasil

fabricio.macedo@aluno.cefet-rj.br, marco.abbud@cefet-rj.br

Abstract. *This work proposes Earendil, an integrated, low-cost and open-source system for automatic pothole detection on urban pavements from drone-captured images. The system comprises four components: an external image source (drone), a backend responsible for ingestion, normalization and persistence, a YOLO26-based inference service, and a web interface that displays georeferenced records on a map. The detector is trained on a unified dataset built from the public UAV-PDD2023 and RDD2022 benchmarks, restricted to aerial capture, and evaluated with standard object detection metrics.*

Resumo. *Este trabalho propõe o Earendil, um sistema integrado, de baixo custo e baseado em componentes de código aberto para a detecção automática de buracos em pavimentos urbanos a partir de imagens capturadas por drone. O sistema é composto por quatro componentes: a fonte externa de imagens (drone), um* backend *responsável pela ingestão, normalização e persistência dos registros, um serviço de inferência baseado na arquitetura YOLO26 e uma interface web que apresenta os registros georreferenciados em um mapa. O detector é treinado sobre um conjunto unificado construído a partir das bases públicas UAV-PDD2023 e RDD2022, restrito ao domínio de captura aérea, e avaliado por métricas consagradas de detecção de objetos.*

1. Introdução

A malha rodoviária é o principal meio de circulação de pessoas e de cargas no Brasil, e sua conservação tem efeito direto sobre a segurança dos usuários e sobre o custo do transporte. A Pesquisa CNT de Rodovias 2025 avaliou 114.197 km de vias pavimentadas e classificou 62,1% da extensão como regular, ruim ou péssima em seu estado geral; segundo o mesmo levantamento, a qualidade do pavimento eleva em média 31,2% os custos operacionais do transporte rodoviário no país (CNT, 2025). O quadro é ainda mais desfavorável nas rodovias sob gestão pública, em que 64,4% da extensão apresenta algum problema de pavimento.

Nas vias urbanas, sob responsabilidade de municípios que frequentemente dispõem de orçamento e corpo técnico reduzidos, o problema é agravado pela ausência de levantamentos periódicos e padronizados equivalentes aos realizados na malha federal. Entre as patologias de pavimentos flexíveis, o buraco, também denominado panela na terminologia normativa (DNIT, 2003), é a que produz efeito mais imediato sobre o usuário: representa estágio avançado de degradação, originado em geral pela evolução de trincas

de fadiga e pela infiltração de água, e está associado a danos em veículos, a desvios bruscos de trajetória e a acidentes, sobretudo com motociclistas e ciclistas.

A avaliação da condição de superfície é hoje conduzida majoritariamente por levantamento visual, procedimento previsto na norma DNIT 008/2003-PRO, no qual técnicos treinados percorrem os trechos registrando tipo, severidade e extensão dos defeitos. Embora de baixo custo unitário, o método é lento, depende do julgamento do avaliador, expõe o inspetor ao tráfego e muitas vezes exige interdição parcial da via. Na prática municipal, esse processo convive com um canal reativo e enviesado: a reclamação do cidadão, que registra apenas os defeitos percebidos por quem decide reclamar e raramente traz evidência visual ou localização confiável.

O uso de veículos aéreos não tripulados (VANTs), popularmente chamados de drones, associado a técnicas de visão computacional, apresenta-se como alternativa promissora. O drone cobre grandes áreas em pouco tempo, alcança trechos de difícil acesso, dispensa a interrupção do tráfego e não expõe o operador ao risco de atropelamento; além disso, registra a posição de captura por meio de GPS embarcado. Sobre essas imagens, detectores de objetos em tempo real da família YOLO (*You Only Look Once*), que processam a imagem inteira em uma única passagem pela rede (Redmon et al., 2016), permitem localizar automaticamente os defeitos com custo computacional compatível com *hardware* modesto.

Há, no entanto, uma lacuna recorrente na literatura nacional sobre o tema: a maior parte dos trabalhos encerra-se no modelo. Treina-se um detector, relatam-se métricas de precisão e revocação em um conjunto de teste e conclui-se pela viabilidade da abordagem, sem que se construa o caminho entre a imagem capturada em campo e a informação efetivamente utilizável por quem decide onde aplicar o recurso de manutenção. Faltam a ingestão automatizada das imagens, a persistência dos registros, o encadeamento com o modelo de inferência e a apresentação dos resultados em uma interface consultável.

Este trabalho ataca precisamente essa lacuna. A questão que orienta a pesquisa é: *é possível construir um sistema integrado, de baixo custo e baseado em componentes de código aberto, capaz de receber imagens de pavimento capturadas por drone, classificar automaticamente a presença de buracos e disponibilizar os registros localizados em uma interface web de consulta, com desempenho de detecção compatível com o relatado na literatura?* A proposta apresentada a seguir, denominada Earendil, responde a essa questão por meio de um sistema de quatro componentes e de um detector treinado sobre dois conjuntos de dados públicos de referência.

2. Solução Proposta

2.1. Objetivo geral

Projetar, implementar e avaliar o Earendil, um sistema baseado em aprendizado de máquina para detecção automática de buracos em pavimentos urbanos a partir de imagens capturadas por drone, contemplando o fluxo completo de captura, processamento, persistência, inferência e visualização georreferenciada dos registros.

2.2. Objetivos específicos

- Construir um conjunto de dados de treinamento unificado a partir dos *datasets* públicos UAV-PDD2023 e RDD2022, harmonizando taxonomias de rótulos e for-

matos de anotação.

- Treinar e avaliar modelos de detecção baseados na arquitetura YOLO26, comparando variantes de tamanho e estabelecendo uma linha de base com versões anteriores da família YOLO.
- Avaliar a capacidade de generalização do modelo entre domínios, treinando em um *dataset* e testando no outro, de modo a estimar o desempenho esperado em imagens não vistas.
- Implementar um serviço de *backend* responsável por receber as imagens enviadas pela fonte externa, normalizá-las, registrá-las em banco de dados e acionar o modelo de inferência.
- Implementar uma interface web que apresente os registros de detecção com a respectiva imagem anotada, nível de confiança, data e posição aproximada obtida do GPS do drone.
- Avaliar o sistema resultante por meio de testes unitários, de integração e de usabilidade, e o modelo por meio de métricas consagradas de detecção de objetos.

2.3. Arquitetura proposta

O Earendil é organizado em quatro componentes com responsabilidades bem delimitadas, comunicando-se por interfaces explícitas. Essa separação permite que o modelo seja substituído ou re-treinado sem alteração dos demais módulos e que a fonte de imagens seja trocada sem impacto no restante do sistema.

Fonte externa (drone). Responsável por registrar as imagens do pavimento e enviá-las ao *backend*. A captura ocorre em voo a baixa altitude sobre a via, com a câmera apontada para a superfície; cada imagem é acompanhada das coordenadas de latitude e longitude fornecidas pelo GPS do equipamento e do instante da captura. O envio é feito por requisição HTTP ao *endpoint* de ingestão do *backend*. Como o objetivo do trabalho é o sistema, e não a automação do voo, esse componente será exercitado tanto por um agente de envio em lote que percorre um diretório de imagens reais de drone e as submete ao *backend* com os metadados correspondentes quanto, quando houver disponibilidade de equipamento, por captura real em campo.

Backend. Núcleo de orquestração do sistema. Recebe a imagem e seus metadados, normaliza o arquivo, atribuindo-lhe um identificador único e um nome padronizado que elimina colisões e preserva a rastreabilidade, logo em seguida armazena o binário no repositório de arquivos e cria o registro correspondente no banco de dados com estado inicial *pendente*. Em seguida, submete a imagem ao modelo de aprendizado de máquina e persiste o resultado: classe atribuída, nível de confiança, caixas delimitadoras e estado final do registro (*confirmado* ou *descartado*). Por fim, expõe os registros por meio de uma API de consulta, consumida pelo *frontend*.

Modelo de aprendizado de máquina. Responsável por classificar a imagem do pavimento, indicando se há buraco e, em caso afirmativo, em que região da imagem. Será implementado com a arquitetura YOLO26, treinada por transferência de aprendizado a partir de pesos pré-treinados e ajustada sobre o conjunto unificado descrito na Seção 4.3. O modelo é exposto ao *backend* como um serviço de inferência, recebendo a imagem e devolvendo a lista de detecções em formato estruturado.

Frontend. Interface web responsável por dar visibilidade aos registros produzi-

dos. Apresenta um mapa com marcadores nas posições informadas pelo GPS do drone e uma listagem tabular filtrável por data, classe e nível de confiança. Ao selecionar um registro, o usuário visualiza a imagem com as caixas delimitadoras sobrepostas e os metadados associados. A localização é tratada de forma aproximada: adota-se diretamente a coordenada registrada pelo drone, sem correção geométrica da projeção nem ortorretificação, o que é suficiente para o propósito de indicar em que trecho da via o defeito se encontra.

2.4. Fluxo de execução

1. O drone sobrevoa a via e captura imagens da superfície do pavimento, associando a cada imagem as coordenadas do GPS e o instante da captura.
2. A fonte externa submete a imagem e os metadados ao *endpoint* de ingestão do *backend*.
3. O *backend* valida a requisição, renomeia e armazena a imagem e cria o registro no banco de dados com estado *pendente*.
4. O *backend* envia a imagem ao modelo, que retorna as detecções encontradas com as respectivas confianças.
5. O *backend* atualiza o registro com o resultado da inferência, marcando-o como *confirmado* quando há detecção acima do limiar de confiança e como *descartado* caso contrário.
6. O *frontend* consulta a API e apresenta os registros no mapa e na listagem, permitindo a inspeção individual de cada ocorrência.

2.5. Contribuições esperadas

A principal contribuição é de natureza sistêmica: entregar, de ponta a ponta, o encadeamento entre captura aérea, detecção automática e visualização, em vez de restringir o trabalho ao treinamento de um detector. Espera-se, adicionalmente, produzir três resultados de interesse: um conjunto de dados unificado a partir de duas bases públicas de referência, com taxonomia harmonizada e documentada; uma avaliação da arquitetura YOLO26, cuja inferência nativamente livre de supressão de não-máximos simplifica a integração com o *backend*, aplicada a imagens aéreas de pavimento, tarefa para a qual ainda há poucos relatos publicados; e um protótipo funcional, licenciado em código aberto, reaproveitável por órgãos municipais e por trabalhos subsequentes.

3. Trabalhos Relacionados

O levantamento a seguir prioriza trabalhos que abordam a detecção automática de defeitos em pavimentos por visão computacional, com atenção especial aos que utilizam imagens aéreas e aos que foram publicados no contexto brasileiro. Ao final, a Tabela 1 sintetiza a comparação e a Seção 3.2 delimita o que a proposta deliberadamente não pretende contemplar.

3.1. Análise comparativa

O trabalho mais próximo da proposta é o de Alzamzami et al. (2024), que apresentam o PDS-UAV, sistema de detecção e gestão de buracos baseado em imagens de drone. Os autores conduziram entrevista com servidor da prefeitura de Jeddah e questionário com 101

usuários das vias para levantar requisitos, modelaram o sistema em UML e implementaram duas aplicações web: uma para os servidores da manutenção viária, que cadastram e atualizam o estado dos buracos, e outra para os usuários da via, que consultam as ocorrências ao longo de uma rota. O detector foi treinado com 1.362 imagens públicas capturadas por drone na Espanha e testado em 111 imagens de Jeddah, em um experimento de generalização entre cidades; entre YOLOv4-tiny, YOLOv5 e YOLOv8, a última obteve o melhor resultado, com precisão de 98%, revocação de 92% e F1 de 95%. A proximidade com o Earendil é evidente na arquitetura (drone, modelo, banco e aplicação web), e o PDS-UAV é adotado aqui como principal referência de projeto.

As diferenças, contudo, são relevantes. No PDS-UAV, o envio das imagens é uma ação manual do servidor, que preenche um formulário com coordenadas e distrito antes de acionar a predição; no Earendil, a ingestão é automatizada por uma interface programática, e a coordenada vem do próprio GPS do equipamento. Enquanto o PDS-UAV utiliza YOLOv8, esta proposta avalia o YOLO26, arquitetura cuja saída direta, sem etapa de pós-processamento por supressão de não-máximos, reduz a latência e simplifica a integração em serviço (Ultralytics, 2026). Por fim, o treinamento do PDS-UAV recorre a um único *dataset* de um só país, ao passo que o Earendil combina uma base aérea multiclasse com uma base multinacional, buscando maior robustez a variações de iluminação, textura e ângulo de captura.

Monte, Goldschmidt e Santos (2025) propõem um modelo híbrido baseado em YOLOv8 para identificação e classificação de patologias em pavimentos flexíveis e rígidos, área em que apontam lacuna após revisão sistemática de 209 estudos publicados entre 2019 e 2024. Coletaram cerca de 4.000 imagens com câmeras GoPro equipadas com GPS em diferentes regiões do Brasil e sob condições climáticas variadas, anotando 1.000 delas manualmente, e relatam mAP@50 próximo de 90% e precisão de até 96%, com validação adicional em 65 segmentos de um quilômetro. É o trabalho brasileiro com melhores métricas entre os analisados e reforça duas escolhas desta proposta: a importância da diversidade de condições de captura e o valor da anotação cuidadosa. A captura, no entanto, é veicular e em nível de solo, e o trabalho concentra-se no modelo, sem descrever a entrega de um sistema de consulta; o *dataset* é proprietário, o que dificulta a reprodução.

Carvalho, Guedes e Figueiredo (2022) tratam a inspeção automática de danos em pavimentação como problema de detecção em visão computacional, no contexto de cidades inteligentes. Avaliam quatro configurações de redes YOLO sobre dados de três países e obtêm mAP de 0,53, com ganhos expressivos no tempo de predição em relação à literatura da época, ainda que com menor número de parâmetros. O estudo de caso conduzido com imagens brasileiras é particularmente instrutivo para esta proposta, pois evidencia a queda de desempenho quando o modelo é aplicado a um domínio distinto do de treinamento, justamente o risco que o teste de generalização entre *datasets*, previsto na Seção 4.5, pretende quantificar.

Batista et al. (2025) avaliam o YOLOv8n sobre um conjunto real de imagens de vias urbanas do Ceará, anotadas com uma única classe, “defeito”, na plataforma Roboflow. Obtêm precisão de 74,60%, revocação de 44,18% e mAP@50 de 0,521, resultado que consideram satisfatório para ocorrências visuais simples. O trabalho é útil como referência de desempenho em contexto urbano brasileiro e ilustra o custo de uma classe única excessivamente ampla: a revocação baixa indica que boa parte dos defeitos presen-

tes não é recuperada. O Earendil concentra-se em uma classe específica e visualmente bem caracterizada, o buraco, na expectativa de obter revocação mais alta.

Wanderley e Silva (2025) investigam a identificação de patologias em pavimentos a partir de imagens de drone com modelos YOLO treinados no Roboflow e executados em Google Colab. Desenvolveram cinco modelos, dos quais quatro foram avaliados: o mais consistente, com 909 imagens de treino e curadoria manual intensiva, alcançou mAP@50 de 55,8%, precisão de 60,6% e revocação de 56,4%; já um modelo treinado de forma automatizada, sem verificação manual das anotações e com base 2,5 vezes maior, caiu para mAP@50 de 29,5%. A conclusão é que a qualidade das anotações é mais determinante que o volume bruto de dados, e portanto é a principal lição incorporada por esta proposta e justifica a decisão de partir de *datasets* de referência revisados por pares, com anotações validadas, em vez de compor uma base própria por coleta oportunista.

Kassem, Matrakas e Bussador (2024) propõem um protótipo baseado em YOLO para detecção de rachaduras em concreto asfáltico, mas o trabalho permanece em estágio de proposta: a construção do banco de imagens, o treinamento e a avaliação são apontados como etapas futuras. O contraste é deliberado nesta seção, pois delimita um risco concreto a ser evitado, o de encerrar o trabalho no desenho da solução. O cronograma da Seção 5 reserva, por isso, blocos explícitos de treinamento, integração e validação.

Guan et al. (2025) apresentam um sistema embarcado de detecção de buracos que combina aprendizado profundo e visão binocular. Constroem um *dataset* de 6.848 imagens com câmera binocular veicular, propõem o modelo GASB-ShuffleNetv2 para reconhecimento do estado do pavimento com acurácia de 98,61% e o PF-CenterNet para localização das regiões afetadas, além de um modelo de estimativa de distância por disparidade estéreo, executado em um Jetson Xavier NX com aceleração por TensorRT e tempo total de 111 ms por imagem. O objetivo é distinto do desta proposta: trata-se de percepção em tempo real para alerta ao condutor, e não de apoio à gestão do patrimônio viário. O trabalho é citado por demonstrar que a inferência pode ser executada em *hardware* embarcado de baixo consumo, o que sustenta a escolha de uma arquitetura orientada à borda como o YOLO26 e abre caminho para uma evolução futura do Earendil com inferência a bordo do próprio drone.

Em síntese, os trabalhos revisados consolidam a viabilidade técnica da detecção automática de buracos por redes da família YOLO, com métricas que variam de mAP@50 em torno de 0,52 em cenários urbanos de classe única até precisão superior a 95% em bases mais controladas. A lacuna que o Earendil ocupa não está, portanto, na detecção em si, mas na integração: entre os trabalhos analisados, apenas o PDS-UAV entrega um sistema operacional baseado em imagens de drone, e mesmo ele depende de submissão manual das imagens e não explora bases de dados aéreas multiclasse nem arquiteturas de detecção ponta a ponta.

3.2. Delimitações do escopo

Para que o trabalho seja concluído no prazo previsto e com profundidade adequada, os seguintes aspectos ficam explicitamente fora do escopo:

- **Georreferenciamento preciso e ortorretificação.** A posição de cada registro será a coordenada bruta informada pelo GPS do drone, sem correção da projeção da

Tabela 1. Comparação entre os trabalhos relacionados e a proposta

Trabalho	Captura	Algoritmo	Dados	Entrega / resultado
Alzamzami et al. (2024): PDS-UAV	Drone (DJI)	YOLOv8	Público (Espanha, 1.362 img.) + 111 img. de Jeddah	Sistema web + modelo; P = 98%, R = 92%, F1 = 95%
Monte et al. (2025)	Câmera GoPro com GPS em veículo	YOLOv8 híbrido	Próprio, ∼4.000 img. (1.000 anotadas)	Modelo; mAP@50 ≈ 90%, P até 96%
Carvalho et al. (2022)	Câmera veicular	YOLO (4 config.)	RDD (3 países) + estudo de caso Brasil	Modelo; mAP ≈ 0,53
Batista et al. (2025)	Câmera em nível de solo	YOLOv8n	Próprio (vias do Ceará), classe única	Modelo; P = 74,6%, R = 44,2%, mAP@50 = 0,521
Wanderley e Silva (2025)	Drone / imagens diversas	YOLO (Roboflow)	Roboflow Universe, ∼1.048 img.	Modelo; mAP@50 = 55,8%, P = 60,6%, R = 56,4%
Kassem et al. (2024)	Imagens digitais de asfalto	YOLO (proposto)	Não construído	Proposta metodológica, sem implementação
Guan et al. (2025)	Câmera binocular embarcada	GASB-ShuffleNetv2 + PF-CenterNet	Próprio, 6.848 img.	Sistema embarcado com estimativa de distância; acurácia 98,6%
Earendil (proposta)	Drone (imagem + GPS)	YOLO26	UAV-PDD2023 + RDD2022 (públicos)	Sistema completo (ingestão, banco, inferência, mapa web) + avaliação

Fonte: elaborado pelo autor, 2026.

imagem, sem geração de ortomosaico e sem uso de pontos de controle no solo. O resultado é uma localização aproximada, adequada para indicar o trecho, não para medir a posição exata do defeito na pista.

- **Quantificação de severidade e de geometria.** O sistema indicará a presença e a posição do buraco na imagem, mas não estimará profundidade, área real em metros quadrados nem grau de severidade segundo a norma DNIT 005/2003-TER, o que exigiria reconstrução tridimensional ou escala conhecida.
- **Detecção do conjunto completo de patologias.** O trabalho concentra-se na classe buraco. Trincas, afundamentos, exsudação e desagregação superficial poderão ser aproveitadas como classes auxiliares no treinamento, mas não serão objeto de avaliação nem de exibição no *frontend*.
- **Inferência a bordo e planejamento autônomo de voo.** A inferência ocorre no servidor, e não no drone; o planejamento de rota, o controle do voo e a conformidade com a regulamentação de operação de VANTs não são tratados.
- **Integração com sistemas de gestão municipal e fluxo de ordens de serviço.** O *frontend* dará visibilidade aos registros, sem controle de estado de reparo, atribuição de equipes ou integração com sistemas legados de prefeituras.
- **Operação em escala de produção.** Não se pretende dimensionar o sistema para uso municipal contínuo, com alta disponibilidade e processamento de grandes volumes; o resultado é um protótipo funcional validado em escala experimental.

4. Metodologia

4.1. Processo de desenvolvimento

O projeto será conduzido por um processo iterativo e incremental, organizado em ciclos de duas semanas. Cada ciclo se encerra com um incremento executável e verificável, o que permite antecipar riscos técnicos, em especial os associados ao desempenho do modelo, e replanejar sem comprometer o cronograma global. A gestão das atividades será feita com auxílio da plataforma de versionamento de código GitHub, e o controle de versão em repositório Git único contendo os módulos de *backend*, *frontend* e treinamento, que poderá ser encontrado no endereço: `https://github.com/JfOliveira2003/Earendil`

A trilha de aprendizado de máquina seguirá, em paralelo, as etapas do CRISP-DM: entendimento do problema e dos dados, preparação, modelagem, avaliação e implantação. A separação das duas trilhas é intencional, pois seus ciclos têm durações distintas (um experimento de treinamento pode levar horas, enquanto um incremento de interface se conclui em minutos) e sua sincronização ocorre nos pontos de integração previstos no cronograma.

4.2. Levantamento de requisitos e modelagem

O levantamento de requisitos combinará três fontes. A primeira é a análise documental das normas técnicas pertinentes, em especial a DNIT 005/2003-TER, que fixa a terminologia dos defeitos, e a DNIT 008/2003-PRO, que descreve o procedimento de levantamento visual, que são insumos necessários para que a nomenclatura adotada no sistema seja compatível com a prática de engenharia. A segunda é a análise de sistemas similares, tomando o PDS-UAV como referência de funcionalidades e organização. A terceira, condicionada à disponibilidade, é uma entrevista semiestruturada com servidor de secretaria municipal de obras ou de infraestrutura, no formato adotado por Alzamzami et al. (2024), destinada a caracterizar o processo atual de identificação e registro de buracos e a validar a utilidade das informações que o *frontend* pretende apresentar.

A modelagem utilizará notação UML, com os seguintes artefatos: diagrama de casos de uso, delimitando as funcionalidades e os atores (operador de captura e consultor dos registros); diagrama de componentes, explicitando as fronteiras entre os quatro módulos e as interfaces que os conectam; diagrama de sequência do fluxo principal, da submissão da imagem à exibição do registro no mapa; e modelo entidade-relacionamento do banco de dados. Os casos de uso serão descritos em formato textual, com pré-condições, fluxo principal e fluxos alternativos, e servirão de base para a definição dos casos de teste.

4.3. Obtenção e preparação dos dados

O treinamento utilizará dois conjuntos de dados públicos, escolhidos por serem revisados por pares, terem anotações validadas e se complementarem em perspectiva de captura. O UAV-PDD2023 (Yan e Zhang, 2023) reúne 2.440 imagens capturadas por drone na China, com mais de 11.150 instâncias anotadas em seis classes de patologia: trincas longitudinais, transversais, oblíquas e interligadas, remendos e buracos, obtidas com câmera DJI Zenmuse H20 em voo a cerca de 30 m de altitude, sob duas condições climáticas e em rodovias de diferentes padrões construtivos, com anotações no formato Pascal VOC. É a base que mais se aproxima da geometria de captura prevista para o Earendil. O RDD2022

(Arya et al., 2024) reúne 47.420 imagens de seis países: Japão, Índia, República Tcheca, Noruega, Estados Unidos e China. Com mais de 55.000 instâncias de dano anotadas em quatro classes, entre elas buraco; o conjunto inclui um subconjunto capturado por drone na China, além de imagens veiculares. Sua contribuição é a diversidade geográfica e de condições de captura, fator que Carvalho et al. (2022) associam diretamente à capacidade de generalização do modelo.

A preparação envolverá quatro etapas. Primeiro, a harmonização de taxonomias: as classes dos dois conjuntos serão mapeadas para um vocabulário comum, com a classe buraco como alvo principal. Segundo, a conversão de formatos: as anotações Pascal VOC do UAV-PDD2023 e as do RDD2022 serão convertidas para o formato de rótulos do YOLO, com normalização das coordenadas das caixas. Terceiro, a filtragem e o balanceamento: serão descartadas imagens sem anotação e instâncias degeneradas, e será avaliado o efeito do desbalanceamento entre classes, dado que buracos são significativamente menos frequentes que trincas em ambas as bases. Quarto, a divisão dos dados em treino, validação e teste na proporção aproximada de 80%, 10% e 10%, com a separação feita por imagem de origem para evitar vazamento entre partições.

Serão aplicadas técnicas de aumento de dados sobre o conjunto de treino (espelhamento horizontal e vertical, rotação, variação de escala, ajuste de brilho e contraste e inserção de ruído), com o objetivo de expor o modelo a diferentes condições de iluminação e ângulo de captura. Seguindo a recomendação de Wanderley e Silva (2025), o aumento será aplicado apenas ao treino, mantendo validação e teste inalterados, e uma amostra das imagens transformadas será inspecionada visualmente para garantir que as caixas delimitadoras permaneçam corretas após cada transformação.

4.4. Pré-processamento dos dados

As etapas de preparação descritas na Seção 4.3 foram materializadas em um único procedimento reprodutível, implementado no *script* `src/merge_datasets.py`. O *script* lê os dois conjuntos de dados diretamente de seus arquivos de distribuição compactados e produz, em uma única passagem, todos os artefatos necessários ao treinamento: as imagens selecionadas, os rótulos no formato YOLO, o arquivo de configuração `data.yml`, um manifesto em CSV com uma linha por imagem exportada e um arquivo `stats.json` com as contagens por fonte e por partição.

A opção por concentrar todo o pré-processamento em um único gerador, em vez de encadear transformações sucessivas sobre um diretório de trabalho, é deliberada. Ela garante que os cinco artefatos de saída sejam mutuamente consistentes por construção, pois o manifesto nunca descreve um conjunto que deixou de existir, e que todo o resultado seja recuperável a partir de uma semente aleatória fixa, requisito de reprodutibilidade já estabelecido na Seção 4.5. Os conjuntos originais permanecem intactos, de modo que variantes do conjunto de treinamento podem ser geradas e comparadas sem risco de perda de dados.

As subseções seguintes descrevem as cinco decisões de pré-processamento adotadas, na ordem em que são aplicadas pelo gerador.

4.4.1. Restrição ao domínio de captura aérea

O Earendil destina-se à inspeção viária por drone, mas o conjunto unificado construído a partir do RDD2022 e do UAV-PDD2023 não representava essa perspectiva. Das 5.062 imagens reunidas na primeira versão do conjunto, apenas 261, cerca de 5%, haviam sido capturadas por VANT; as demais provinham de câmeras embarcadas em automóveis e motocicletas. As duas perspectivas não constituem variações menores de um mesmo problema de visão, como sintetiza a Tabela 2.

Tabela 2. Diferenças entre as perspectivas de captura

Aspecto	Captura em nível de solo	Captura por drone
Atitude da câmera	Oblíqua, com o horizonte no quadro	Nadir (perpendicular ao solo)
Cobertura da via	Terço inferior da imagem	Imagem inteira
Conteúdo da cena	Céu, edificações, tráfego, vegetação	Apenas a textura do pavimento
Aparência do buraco	Pequeno, escorçado, distorcido pela perspectiva	Quase planar, em escala consistente
Iluminação	Mista, com sombras projetadas intensas	Comparativamente uniforme

Fonte: elaborado pelo autor, 2026.

Um detector treinado majoritariamente sobre quadros oblíquos de câmera veicular aprende uma geometria de projeção e um contexto de cena que não ocorrem em imagens nadir de drone. Manter essa maioria no conjunto de treinamento equivaleria a otimizar o modelo para uma condição de operação que não é a do sistema proposto. O conjunto foi, por isso, restringido às fontes aéreas.

O UAV-PDD2023 é integralmente capturado por drone e qualifica-se por completo. O RDD2022, por sua vez, é organizado por país, e os países diferem quanto à plataforma de captura: seis dos sete subconjuntos (Índia, Japão, República Tcheca, Noruega, Estados Unidos e `China_MotorBike`) são veiculares, ao passo que `China_Drone` é aéreo, o que se confirma pela inspeção de suas imagens: quadros de 512×512 pixels do pavimento vistos de cima, sem horizonte e sem contexto de borda de via, visualmente compatíveis com o material do UAV-PDD2023. A restrição não consiste, portanto, em descartar o RDD2022, mas em preservar seu subconjunto aéreo e dispensar os seis subconjuntos de nível de solo.

O filtro é aplicado no momento da indexação, e não na exportação. A distinção tem consequência prática relevante: os seis arquivos excluídos sequer chegam a ser abertos, apenas o da Noruega ocupa 10,6 GB, e a geração completa do conjunto aéreo conclui-se em segundos. O comportamento é controlado pela opção `--drone-only`, desativada por padrão, de modo que o conjunto misto original permanece reprodutível a partir do mesmo *script*, condição necessária para que a comparação entre os dois regimes de treinamento seja justa.

4.4.2. Harmonização de taxonomias e conversão de formatos

As duas bases nomeiam o buraco de maneiras distintas: o RDD2022 o anota sob o código `D40` e o UAV-PDD2023 sob o rótulo `Pothole`. Ambos são mapeados para a classe única `0 Pothole` do conjunto unificado. As demais categorias de patologia presentes nas bases, isto é, trincas longitudinais, transversais, oblíquas e interligadas, além de remendos, são descartadas em vez de reaproveitadas como classes auxiliares. A consequência dessa decisão é que uma imagem que contenha apenas trincas não é removida do conjunto: ela passa a constituir um exemplo negativo, isto é, uma amostra de pavimento sem buraco, o que é coerente com o escopo delimitado na Seção 3.2, em que apenas a classe buraco é objeto de avaliação.

As anotações originais, ambas no formato Pascal VOC, são convertidas para o formato de rótulos do YOLO, com as coordenadas do centro, da largura e da altura de cada caixa normalizadas em relação às dimensões da imagem. Três cuidados foram incorporados à conversão:

- **Dimensões obtidas da própria imagem.** A normalização utiliza a largura e a altura reais do arquivo JPEG decodificado, e não o campo `<size>` registrado no XML, que se mostrou inconsistente em parte das anotações do RDD2022. Normalizar por uma dimensão incorreta produziria caixas deslocadas de forma silenciosa, sem qualquer erro de execução.
- **Recorte às bordas e correção de orientação.** Coordenadas invertidas são reordenadas e caixas que extrapolam os limites da imagem são recortadas a eles.
- **Descarte de caixas degeneradas.** Caixas com menos de dois pixels em qualquer dos lados após o recorte são eliminadas, por não representarem objeto anotável e por degradarem o cálculo da função de perda.

Imagens para as quais não existe anotação correspondente no arquivo, bem como arquivos truncados ou ilegíveis, são registrados em *log* e ignorados.

4.4.3. Seleção de exemplos negativos

Todas as imagens que contêm ao menos um buraco são retidas. A elas somam-se exemplos negativos na proporção de 0,33 negativo por positivo, amostrados aleatoriamente sob a semente fixa. A presença controlada de negativos expõe o detector a pavimento íntegro e a patologias que não são buraco, reduzindo a tendência a predizer em excesso; a proporção é mantida deliberadamente abaixo da paridade para não diluir o sinal positivo em um conjunto já reduzido.

A razão é calculada dentro de cada fonte, e não sobre o agregado, de modo que o equilíbrio entre positivos e negativos se verifique em cada base isoladamente e não apenas na soma. Cada imagem negativa recebe um arquivo de rótulo vazio, convenção pela qual o YOLO a interpreta como imagem sem objetos, em vez de descartá-la por ausência de anotação.

4.4.4. Partição em treino, validação e teste

O UAV-PDD2023 distribui, junto das fotografias originais, cópias já aumentadas de cada uma delas: espelhamentos horizontal e vertical e rotação de $180°$, identificados pelos prefixos `lr_`, `tb_` e `r180_`, e recortes por quadrante, identificados por sufixos como `_top_left`. Tratar esses arquivos como amostras independentes seria um erro metodológico grave: cópias quase idênticas de uma mesma fotografia se distribuiriam entre treino e validação, e o modelo seria validado sobre imagens que já viu, inflando artificialmente a métrica.

A partição é feita, por isso, sobre grupos e não sobre arquivos. Todas as cópias derivadas de uma mesma fotografia de origem constituem um grupo e são necessariamente atribuídas à mesma partição. Pela mesma razão, as listas oficiais de partição distribuídas pelo UAV-PDD2023 (`ImageSets/Main`) não foram utilizadas: a verificação mostrou que 184 de seus 185 grupos de validação ocorrem também no conjunto de treino. As imagens do RDD2022 são independentes entre si e cada uma constitui seu próprio grupo.

A atribuição é ainda estratificada pelo par (fonte, presença de buraco), de modo que ambas as fontes e ambas as condições estejam representadas em validação e em teste, e emprega alocação por maiores restos para que nenhuma partição seja silenciosamente reduzida a zero em estratos pequenos. As proporções adotadas são de 80% para treino, 10% para validação e 10% para teste.

4.4.5. Aumento de dados

O aumento de dados é aplicado exclusivamente ao conjunto de treino, em tempo de execução, pelo *pipeline* da biblioteca Ultralytics, que a cada época sorteia transformações de espelhamento, escala, translação e ajuste de matiz, saturação e brilho sobre as imagens carregadas, propagando a transformação correspondente às caixas delimitadoras. Os conjuntos de validação e teste não recebem qualquer transformação, conforme a recomendação de Wanderley e Silva (2025) incorporada na Seção 4.3.

Optou-se por não gerar cópias aumentadas em disco durante o pré-processamento. O aumento em tempo de execução expõe o modelo a uma variação distinta a cada época, em vez de a um conjunto fixo de cópias, e, o que é determinante neste caso, não cria novos arquivos cuja filiação a uma fotografia de origem precisaria ser rastreada pelo mecanismo de grupos descrito na Seção 4.4.4, preservando a integridade da partição.

4.4.6. Conjunto resultante

O conjunto aéreo resultante é gravado em `dataset/drone/`, preservado o conjunto misto em `dataset/merged/` para efeito de comparação. Sua composição é apresentada nas Tabelas 3 e 4.

A restrição preserva a totalidade do material aéreo disponível: as 281 caixas de buraco correspondem a todas as que existem nas duas bases em imagens capturadas por drone (195 no UAV-PDD2023 e 86 no `China_Drone`), de modo que nada se perdeu além do material de nível de solo que o filtro se destinava a remover.

Tabela 3. Composição do conjunto aéreo, por fonte

Fonte	Imagens	Positivas	Negativas	Caixas	Resolução
UAV-PDD2023	176	132	44	195	2592×1944
RDD2022 (`China_Drone`)	85	64	21	86	512×512
Total	**261**	**196**	**65**	**281**	

Fonte: elaborado pelo autor, 2026.

Tabela 4. Distribuição por partição

Partição	Imagens	Imagens positivas	Caixas	Grupos
Treino	210	159	228	120
Validação	28	21	28	14
Teste	23	16	25	16

Fonte: elaborado pelo autor, 2026.

A integridade do conjunto gerado foi verificada por quatro checagens automatizadas: que apenas fontes aéreas figuram no manifesto; que a contagem total de caixas corresponde à esperada; que nenhum grupo de aumento se distribui entre duas partições; e que todos os rótulos são sintaticamente válidos, com identificador de classe igual a zero e coordenadas normalizadas no intervalo adequado. Todas as checagens são satisfeitas pelo conjunto em uso.

Como o diretório de dados não é versionado (os arquivos de distribuição e os conjuntos gerados somam dezenas de gigabytes), o registro reprodutível do conjunto é o próprio gerador, acompanhado da semente e dos parâmetros registrados em `stats.json`.

4.4.7. Limitações decorrentes do pré-processamento

Duas consequências da restrição ao domínio aéreo devem ser declaradas junto de qualquer resultado obtido sobre este conjunto.

O conjunto é pequeno e sua diversidade efetiva é menor que a contagem de imagens sugere. As 132 imagens positivas do UAV-PDD2023 derivam de apenas 31 fotografias distintas; as demais são espelhamentos, rotações e recortes dessas 31. Somadas aos 64 quadros independentes do `China_Drone`, o conjunto contém aproximadamente 95 cenas de buraco genuinamente independentes. Trata-se de um teto imposto pelas bases de origem, e não pelo filtro: o UAV-PDD2023 contém 195 caixas de buraco contra 5.398 de trinca transversal, e o `China_Drone` fornece 86, todas já incluídas.

Os conjuntos de validação e teste são reduzidos. A partição deixa 21 imagens positivas de validação, oriundas de 9 cenas independentes, e 16 imagens positivas de teste, oriundas de 10. Nessa escala, uma única imagem desloca o mAP em vários pontos percentuais, de modo que a métrica de validação oscilará apreciavelmente entre épocas e o critério de parada antecipada repousará sobre sinal fraco. Os valores obtidos sobre o conjunto de teste devem ser reportados com essa ressalva e não comparados aos do conjunto misto como se houvessem sido medidos com precisão equivalente.

Ambas as limitações são propriedades dos dados aéreos disponíveis publicamente.

Caso se mostrem impeditivas, três mitigações estão previstas: elevar a proporção de negativos, aproveitando as cerca de 4.645 imagens aéreas sem buraco ainda não utilizadas nas duas bases; adotar uma divisão mais larga, da ordem de 70/15/15; ou substituir a partição única por validação cruzada sobre os grupos, o que produz estimativa mais estável ao custo de múltiplos treinamentos.

4.5. Treinamento e avaliação do modelo

O detector será implementado com a arquitetura **YOLO26**, da biblioteca Ultralytics, por três razões. A primeira é a inferência nativamente ponta a ponta: o modelo produz as detecções finais diretamente, sem a etapa de supressão de não-máximos exigida por versões anteriores, o que reduz a latência e elimina uma fonte de divergência de comportamento entre plataformas, vantagem operacional direta para um serviço de inferência acionado pelo *backend*. A segunda é a orientação a dispositivos de borda e a execução eficiente em CPU, característica que viabiliza a implantação do protótipo sem GPU dedicada e habilita a evolução futura para inferência embarcada. A terceira são as melhorias reportadas na detecção de objetos pequenos, decorrentes das funções de perda ProgLoss e STAL, aspecto crítico em imagens aéreas, nas quais o buraco ocupa fração reduzida do quadro (Ultralytics, 2026).

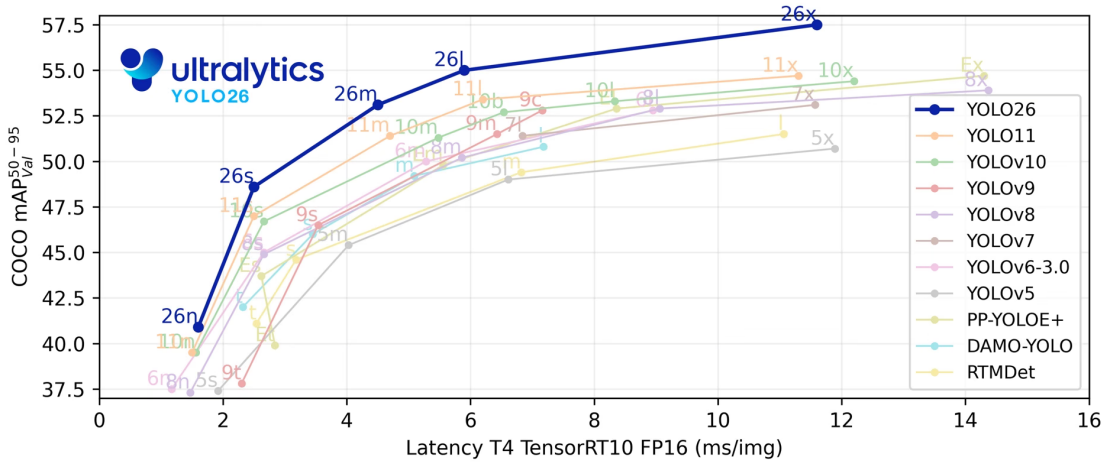


Figura 1. Comparação de desempenho entre o YOLO26 e arquiteturas anteriores de detecção de objetos

Fonte: Ultralytics (2026).

O treinamento empregará transferência de aprendizado a partir de pesos pré-treinados em base de propósito geral, estratégia que reduz o tempo de convergência e melhora a generalização em conjuntos de tamanho moderado. Serão treinadas as variantes *nano* e *small* da arquitetura, com resolução de entrada de 640 pixels a princípio, e os hiperparâmetros de taxa de aprendizado, tamanho de lote e número de épocas serão ajustados a partir do comportamento das curvas de perda no conjunto de validação. Como linha de base comparativa, modelos YOLOv8n e YOLO11n serão treinados sobre exatamente o mesmo conjunto e protocolo, de modo a isolar o efeito da arquitetura. Os experimentos serão executados em ambiente com aceleração por GPU, em Google Colab ou em máquina local equivalente, com registro sistemático de configuração, sementes aleatórias e artefatos de cada execução para assegurar reprodutibilidade.

A avaliação do modelo empregará as métricas consagradas em detecção de objetos (Padilla et al., 2020): mAP@50 e mAP@50-95, precisão, revocação e F1, calculadas sobre o conjunto de teste, que não participa de nenhuma etapa de treinamento. Serão medidos também o tempo de inferência por imagem em CPU e em GPU e o tamanho do modelo em disco, indicadores relevantes para a decisão de implantação. A análise dos erros será qualitativa além de quantitativa, com inspeção das imagens que geraram falsos positivos e falsos negativos, para identificar os confundidores recorrentes já apontados na literatura, como remendos, manchas de óleo, sombras intensas e bueiros.

4.6. Tecnologias e justificativas

Modelo e treinamento. Python, com PyTorch e a biblioteca Ultralytics, que fornece implementação de referência do YOLO26 e das versões anteriores usadas como linha de base, além de rotinas padronizadas de avaliação, condição para que a comparação entre arquiteturas seja justa.

Backend. Python com FastAPI, escolhido pela validação automática de dados de entrada, pelo suporte nativo a operações assíncronas (útil no tratamento de envios concorrentes de imagens) e pela geração automática de documentação da API, que facilita o desenvolvimento independente do *frontend*. Manter o *backend* na mesma linguagem do modelo elimina a necessidade de uma camada de comunicação entre processos em linguagens distintas. A persistência usará MariaDB, por sua maturidade no tratamento de tipos numéricos e temporais e pela disponibilidade de extensões geoespaciais caso a evolução do projeto as demande, com acesso via *shell* (linha de comando).

Frontend. HTML, CSS e JavaScript, pela simplicidade de implantação e pela ausência de etapa de compilação, suficientes para a interface de consulta prevista.

Infraestrutura e qualidade. Todos os serviços serão empacotados em contêineres Docker e orquestrados por Docker Compose, garantindo reprodutibilidade do ambiente e simplicidade de implantação. Os testes automatizados serão escritos com *pytest* no *backend*, e uma esteira de integração contínua no GitHub Actions executará a suíte a cada alteração.

4.7. Testes e validação

A verificação do sistema ocorrerá em quatro níveis. Os testes unitários cobrirão as funções de normalização e renomeação de arquivos, validação de metadados, cálculo de limiares e serialização das detecções. Os testes de integração exercitarão o fluxo completo, da submissão da imagem à consulta do registro pela API, com banco e repositório de arquivos efêmeros, e verificarão os casos de erro: imagem corrompida, formato não suportado, coordenadas ausentes ou inválidas e indisponibilidade do serviço de inferência. Um teste de carga leve medirá o tempo de resposta e a taxa de falhas sob envio concorrente de imagens, estabelecendo o limite prático de operação do protótipo. Por fim, uma validação ponta a ponta submeterá ao sistema um lote de imagens reais de drone não pertencentes ao conjunto de treinamento, comparando as detecções produzidas com a anotação manual das mesmas imagens.

4.8. Critérios de sucesso

O projeto será considerado bem-sucedido se satisfizer os seguintes critérios, verificáveis de forma objetiva:

- **Funcional.** O fluxo completo opera de ponta a ponta, sem intervenção manual entre a submissão da imagem pela fonte externa e a exibição do registro no *frontend*, com os quatro componentes integrados e implantáveis por um único comando.
- **Desempenho do modelo.** mAP@50 igual ou superior a 0,65, precisão igual ou superior a 0,75 e revocação igual ou superior a 0,70 para a classe buraco no conjunto de teste, patamares situados acima dos relatados por Batista et al. (2025) e por Wanderley e Silva (2025) em contexto brasileiro, e abaixo dos obtidos em bases mais controladas, o que os torna simultaneamente ambiciosos e alcançáveis.
- **Desempenho do sistema.** Tempo mediano de processamento por imagem inferior a um segundo em CPU, medido entre o recebimento da requisição e a persistência do resultado.
- **Qualidade e usabilidade.** Cobertura de testes automatizados igual ou superior a 70% nos módulos de *backend* e pontuação SUS igual ou superior a 68, valor de referência para usabilidade adequada.

O não atingimento dos patamares de desempenho do modelo não invalida o trabalho: nesse caso, a análise das causas (desbalanceamento de classes, limitações das bases quanto a pavimentos urbanos brasileiros ou inadequação da resolução de entrada ao tamanho aparente dos defeitos) passa a constituir resultado da pesquisa, desde que os demais critérios sejam satisfeitos.

5. Cronograma

As atividades estão distribuídas entre Projeto Final I, dedicado à fundamentação, à modelagem, à preparação dos dados e aos primeiros experimentos de treinamento, e Projeto Final II, voltado ao refinamento do modelo, à construção do *frontend*, à integração e à validação. Nenhuma atividade havia sido executada até a elaboração desta proposta.

Tabela 5. Cronograma de atividades

Atividade	M1	M2	M3	M4	M5	M6	M7	M8
Revisão bibliográfica	•	•	•					
Levantamento de requisitos e modelagem UML	•	•						
Preparação do conjunto de dados		•	•					
Primeiros experimentos de treinamento			•	•				
Implementação do *backend*			•	•	•			
Refinamento do modelo					•	•		
Implementação do *frontend*					•	•		
Integração dos componentes						•	•	
Testes e validação ponta a ponta							•	•
Redação da monografia				•	•	•	•	•

Fonte: elaborado pelo autor, 2026.

Referências

Alzamzami, O., Babour, A., Baalawi, W., & Al Khuzayem, L. (2024). PDS-UAV: A deep learning-based pothole detection system using unmanned aerial vehicle images. *Sustainability*, 16(21), 9168. https://doi.org/10.3390/su16219168

Arya, D., Maeda, H., Sekimoto, Y., Omata, H., Ghosh, S. K., Toshniwal, D., et al. (2024). RDD2022: A multi-national image dataset for automatic road damage detection. *Geoscience Data Journal*. https://doi.org/10.1002/gdj3.260

Batista, N. S., Oliveira, V. C., Souza, T. I. A., Castro, M. T., Gomes, D. G., Bertoncini, B. V., & Branco, V. T. F. C. (2025). Detecção automática de defeitos em pavimentos: uma abordagem com *deep learning* via YOLOv8n. Em *Anais do XIX Brazilian e-Science Workshop* (pp. 57-64). Porto Alegre: SBC. https://doi.org/10.5753/bresci.2025.248214

Carvalho, R. B., Guedes, E. B., & Figueiredo, C. M. S. (2022). Detecção inteligente de falhas em pavimentações asfálticas com redes neurais convolucionais regionais. Em *Anais do III Workshop Brasileiro de Cidades Inteligentes* (pp. 119-130). Porto Alegre: SBC. https://doi.org/10.5753/wbci.2022.222795

CNT (2025). *Pesquisa CNT de Rodovias 2025*. Brasília: Confederação Nacional do Transporte.

DNIT (2003). *DNIT 005/2003-TER: Defeitos nos pavimentos flexíveis e semirrígidos. Terminologia*. Rio de Janeiro: Departamento Nacional de Infraestrutura de Transportes.

DNIT (2003). *DNIT 008/2003-PRO: Levantamento visual contínuo para avaliação da superfície de pavimentos flexíveis e semirrígidos. Procedimento*. Rio de Janeiro: Departamento Nacional de Infraestrutura de Transportes.

Guan, T., Cai, J., Wang, Y., Yang, W., Chang, X., & Han, Y. (2025). Pavement pothole detection system based on deep learning and binocular vision. *Journal of Traffic and Transportation Engineering (English Edition)*, 12(4), 1100-1123. https://doi.org/10.1016/j.jtte.2024.08.001

Kassem, A. R. K., Matrakas, M. D., & Bussador, A. (2024). Algoritmo de detecção de rachaduras no concreto asfáltico betuminoso por meio de visão computacional. Em *Anais do XXI Congresso Latino-Americano de Software Livre e Tecnologias Abertas* (pp. 356-359). Porto Alegre: SBC. https://doi.org/10.5753/latinoware.2024.245330

Monte, D. M. F. M. do, Goldschmidt, R. R., & Santos, M. dos (2025). Identificação e classificação de patologias em pavimentos rodoviários flexíveis e rígidos utilizando YOLOv8. Em *Proceedings of the 31st Brazilian Symposium on Multimedia and the Web* (pp. 293-301). Porto Alegre: SBC. https://doi.org/10.5753/webmedia.2025.16131

Padilla, R., Netto, S. L., & da Silva, E. A. B. (2020). A survey on performance metrics for object-detection algorithms. Em *Proceedings of the International Conference on Systems, Signals and Image Processing (IWSSIP)* (pp. 237-242). Niterói: IEEE.

Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. Em *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 779-788). Las Vegas: IEEE.

Ultralytics (2026). *Ultralytics YOLO26*. Recuperado de https://docs.ultralytics.com/models/yolo26/

Wanderley, B. P., & Silva, P. B. da (2025). Análise de patologia de pavimentos através de fotos capturadas por drone utilizando IA. Em *Anais da XXI Jornada de Iniciação Científica*. São Paulo: Universidade Presbiteriana Mackenzie.

Yan, H., & Zhang, J. (2023). UAV-PDD2023: A benchmark dataset for pavement distress detection based on UAV images. *Data in Brief*, 51, 109692. https://doi.org/10.1016/j.dib.2023.109692